

Tasky
System Design Document
Versione 0.4



Data: 19/11/2025

Progetto: Tasky	Versione: 0.4
Documento: System Design Document	Data: 19/11/2025

Coordinatore del progetto:

Nome	Matricola
Luca Vincenzo Lambiase Fontana	0512113338

Partecipanti:

Nome	Matricola
Ilaria Lastra	0512120280
Rinaldo Perna	0512119593

Scritto da:	Ilaria Lastra
--------------------	---------------

Revision History

Data	Versione	Descrizione	Autore
01/10/2025	0.1	Presentazione Proposta di Progetto	Lastra Ilaria
10/10/2025	0.2	Presentazione Problem Statement con descrizione degli scenari principali dell'applicazione, i suoi requisiti e gli ambienti target	Lastra Ilaria
20/10/2025	0.3	Presentazione del Requirements Analysis Document, con perfezionamento dei requisiti esposti nel Problem Statement e definizione dei criteri di successo e dello scopo e ambito del sistema	Lastra Ilaria
08/11/2025	0.3.1	Presentazione dell'Object Model e del Dynamic Model, con Class Diagram, Sequence Diagram e Statechart Diagram relativi al Requirements Analysis Document	Lastra Ilaria
19/11/2025	0.4	Presentazione del System Design Document, con discussione dei design goals, dell'architettura e mappatura hardware, software.	Lastra Ilaria, Luca Vincenzo Lambiase Fontana, Rinaldo Perna

Sommario

1.	Introduzione.....	5
1.1.	Purpose of the system.....	5
1.2.	Design goals	6
1.3.	Definitions, acronyms and abbreviations	6
1.4.	References	7
1.5.	Overview	7
2.	Current software architecture	7
3.	Proposed software architecture	8
3.1	Overview.....	9
3.2	Subsystem decomposition	9
3.3	Hardware/software mapping	10
3.4	Persistent data management.....	10
3.5	Access control and security	11
3.5.1	Access Matrix.....	11
3.5.2	Security Strategy.....	12
3.6	Global software control.....	13
3.6.1	Concurrency Management	13
3.7	Boundary conditions	14
4.	Subsystem services	15
5.	Glossary	17

1. Introduzione

1.1. Purpose of the system

Il sistema **Tasky** è una piattaforma di task management aziendale il cui scopo primario è **automatizzare e standardizzare il processo di gestione dei compiti (task) e dei progetti** all'interno di grandi aziende di sviluppo.

Attualmente, l'organizzazione del lavoro si basa su meeting in presenza e annotazioni manuali, un approccio inefficiente che comporta:

- **Difficoltà per i Manager:** A causa della mancanza di automazione, i Manager riscontrano serie difficoltà nel tenere traccia dell'organizzazione di numerosi progetti, nella suddivisione dei task e nel monitoraggio dello stato di avanzamento assegnato all'intero dipartimento.
- **Rischio di Errore Umano:** Le decisioni non automatizzate possono portare a errori di natura umana nelle annotazioni.
- **Perdita di Chiarezza per i Dipendenti:** I Dipendenti possono facilmente confondersi o dimenticare le scadenze dei task e il lavoro svolto, specialmente se sono impegnati in svariate parti di un progetto.
- **Mancanza di Tracciabilità:** Lo stato di un task non è sempre immediatamente chiaro al Manager.

L'obiettivo nello sviluppo di Tasky è creare una piattaforma **"ad hoc"** che contrasti questa inefficienza, fornendo un servizio che risponde alle esigenze di tutti i membri di una grande società.

Nello specifico, il sistema si prefigge di:

1. **Supporto al Management:** Fornire ai Manager gli strumenti per registrare, suddividere e delegare i task per un progetto, mantenendo una chiara visibilità sulla progressione del lavoro e sullo stato di tutti i task assegnati ai Dipendenti del proprio dipartimento.
2. **Chiarezza Operativa per i Dipendenti:** Consentire ai Dipendenti di visualizzare in modo immediato tutti i task a loro assegnati, le relative descrizioni e le scadenze, facilitando la programmazione dei propri impegni.
3. **Comunicazione dello Stato:** Permettere ai Dipendenti di notificare il completamento dei task ai Manager tramite l'aggiornamento dello stato.
4. **Gestione delle Scadenze:** Implementare un meccanismo automatico che garantisca il rispetto delle scadenze, modificando lo stato di un task in **"Sospesa"** se la data di fine viene superata.

In sintesi, Tasky fornirà una soluzione per l'organizzazione del lavoro che assicurerà **chiarezza, sicurezza e robustezza** alla gestione dei compiti aziendali.

1.2.Design goals

Design Goal	ID	Descrizione	Categoria	Origine
Reliability and Fault Tolerance	DG_1	Il sistema deve garantire operatività completa 24 ore su 24 e prevenire problemi legati alla disconnessione (ConnectionTimeoutException), con un <i>mean time of failure</i> ideale di un anno. Deve anche eseguire backup completi del database periodicamente	Affidabilità (Reliability)	RNF3.3.2
High Performance and Efficiency	DG_2	La schermata principale e il login devono avvenire entro 2 secondi e le liste devono essere visualizzate entro 2 secondi. Le notifiche di feedback devono apparire in meno di un secondo	Performance	RNF3.3.3
Modifiability and Maintainability	DG_3	Il sistema deve essere progettato per facilitare aggiornamenti e modifiche future , utilizzando codice sorgente organizzato in moduli ben definiti . (Questo è supportato dall'architettura Three Tier).	Supportability	RNF3.3.4
Usability and Ease of Use	DG_4	Le funzionalità chiave devono essere facilmente accessibili e comprensibili all'utente . Il sistema deve guidare l'utente nell'inserimento di input con <i>warning</i> chiari in caso di errore	Usability	RNF3.3.1
Portability	DG_5	L'obiettivo è l'estensione successiva a dispositivi iOS , dopo lo sviluppo iniziale per Android	Supportability	RNF3.3.4

1.3.Definitions, acronyms and abbreviations

TERMINE	DEFINIZIONE
RNF	Requisito Non Funzionale. Un vincolo di qualità o prestazione imposto al sistema, come Sicurezza, Affidabilità o Usabilità
DG	Design Goal. Obiettivi per l'identificazione delle qualità che il sistema deve ottimizzare, in riferimento ai requisiti non funzionali individuati in fase di analisi.
DBMS	DataBase Managment System, è un software progettato per gestire in modo efficiente l'archiviazione, il recupero, la manipolazione e la sicurezza dei dati all'interno di un Database
ACID	Acronimo che definisce l'insieme di proprietà fondamentali possedute dalle transazioni del database (Atomicità, Coerenza, Isolamento e Durabilità)

DAL	Data Access Level, è un sottosistema logico o strato di astrazione software il cui scopo è fornire l'accesso ai dati persistenti di un'applicazione
CRUD	Un acronimo che definisce le quattro operazioni di base necessarie per manipolare i dati in modo persistente (Create, Read, Update, Delete)
RBAC	Un modello di sicurezza ampiamente adottato, che definisce i permessi di un utente non individualmente, ma in base al Ruolo che ricopre all'interno dell'organizzazione
API RESTful	Un'interfaccia di programmazione applicativa (API) che aderisce ai principi di progettazione dell'architettura REST (Representational State Transfer) .

1.4. References

- Università degli Studi di Salerno – Corso di Ingegneria del Software – “Proposta Progetto Tasky”
- Università degli Studi di Salerno – Corso di Ingegneria del Software – “Problem Statement Tasky”
- Università degli Studi di Salerno – Corso di Ingegneria del Software – “Requirements Analysis Document Tasky”

1.5. Overview

Tasky Application è un sistema di **Task Management aziendale** sviluppato per dispositivi mobili, progettato per strutturare e monitorare l'esecuzione di progetti complessi e le tasks che li compongono, all'interno di specifici Dipartimenti. Il sistema implementa una chiara distinzione di ruolo: i **Manager** supervisionano, gestiscono e assegnano il lavoro, mentre i **Dipendenti** sono responsabili dell'esecuzione e dell'aggiornamento dello stato dei Task. Gli obiettivi principali del sistema sono elencati dai **Design Goals** definiti, ossia dagli aspetti che si vogliono perseguire per fornire una certa qualità al nostro prodotto. Punti focali riguardano la volontà di garantire una certa **reliability** e **fault tolerance**, caratteristiche peculiari di **performance** e di **efficienza** nei confronti del caricamento delle risorse richieste e soprattutto **usabilità**, **portabilità** per future espansioni ai dispositivi iOS e **modifiability** per l'implementazione di future funzionalità aggiuntive a supporto delle attuali e dell'utenza target di Tasky. Il sistema opera in un'architettura che sappia soprattutto gestire le problematiche e gli svantaggi legati alle **attuali proposte di mercato**, come presentato di seguito.

2.Current software architecture

Tasky si propone di **integrare o migliorare** l'offerta di **applicativi** volti alla gestione del lavoro aziendale, colmando le **lacune** attualmente **presenti** sul mercato.. Il principale competitor che ha ispirato (o verso il quale ci differenziamo) la creazione della nostra applicazione è **Trello**, la cui concezione è **simile** al modello su cui si basa **Tasky**. **Trello** è una piattaforma di **project management** basata su **cloud** (o **SaaS - Software as a Service**) che aiuta individui e team a organizzare progetti e attività in modo **visivo e intuitivo**. Il suo principio fondamentale è l'utilizzo di **board** virtuali (lavagne) per offrire una chiara rappresentazione dello stato di avanzamento.

La struttura si articola in tre elementi principali:

- **Board (Bacheca/Lavagna):** Rappresenta il progetto o il tema principale.
- **Liste:** Contenute all'interno delle board, servono a suddividere il flusso di lavoro in fasi o categorie (es. "Da fare", "In corso", "Completato").
- **Card (Schede):** Inserite nelle liste, rappresentano le **singole attività (task)** e possono includere dettagli come **descrizioni**, **checklist**, **date di scadenza**, **commenti** e **assegnazioni**

a membri specifici del team, garantendo flessibilità.

Questo sistema promuove la **collaborazione**, migliora la **trasparenza** e semplifica la gestione delle priorità. L'accesso base è fornito tramite la creazione di un account **gratuito**, permettendo ai membri del team di visualizzare le board in **tempo reale**.

Per utilizzare Trello bisogna creare un account gratuito e poter usufruire delle funzionalità appena descritte. Vi è anche la possibilità di permettere a tutti i membri del team di visualizzare le board in tempo reale. **Trello presenta tuttavia alcuni limiti e svantaggi nell'utilizzo, che Tasky si propone di affrontare:**

1. **Scalabilità e Complessità:** Il principale limite emerge nella gestione di **progetti di grandi dimensioni** o **altamente complessi**. Quando una board accumula un numero eccessivo di card, la visualizzazione può diventare rapidamente **dispersiva e caotica**, compromettendo la capacità di mantenere una **visione d'insieme** senza ricorrere a filtri o strumenti esterni.
2. **Limitazioni della Versione Gratuita (Freemium Model):** Per sbloccare funzionalità avanzate cruciali per i team professionali, come l'**automazione dei task** (tramite Butler) o l'uso **illimitato** dei **Power-Ups** (integrazioni di terze parti), è necessario sottoscrivere **piani a pagamento (Premium)**. Questo costituisce un vincolo significativo, soprattutto per i **team più numerosi** o con esigenze operative articolate.
3. **Necessità di Piani Superiori:** In sintesi, i team con processi complessi o di grandi dimensioni sono di fatto **obbligati** ad adottare i piani a pagamento per sfruttare appieno il potenziale della piattaforma.

Dal punto di vista architettonico, Trello è implementato con un modello client-server, tipico delle applicazioni web based e cloud native. Le versioni mobile (iOS/Android), pur permettendo la gestione dei progetti, tradizionalmente offrono un set di funzionalità e servizi meno completo rispetto alla versione web/desktop.

3. Proposed software architecture

La **Tasky Application** è una soluzione software robusta e scalabile per il **Task Management aziendale**, specificamente sviluppata per operare su **piattaforme mobili** in un ambiente ad alta affidabilità. Il sistema è progettato per strutturare e monitorare l'esecuzione di **progetti complessi** e le **unità di lavoro minime** (Task) all'interno di **specifici Dipartimenti**.

La struttura operativa è definita da una chiara distinzione di ruolo basata su **Role-Based Access Control (RBAC)**:

- Il **Manager** detiene privilegi amministrativi, gestionali e di supervisione su Progetti, Task e membri del Dipartimento.
- Il **Dipendente** è l'esecutore diretto, responsabile dell'esecuzione e dell'aggiornamento limitato dello stato dei Task a lui assegnati.

Gli obiettivi principali del sistema sono guidati dai **Design Goals** (evinti dai Requisiti Non Funzionali – RNF definiti nella fase di analisi). Punti focali riguardano la volontà di garantire un'elevata **Reliability** (Affidabilità) e **Fault Tolerance**, caratteristiche peculiari di **Performance** ed **Efficienza** nei confronti del caricamento delle risorse richieste. Inoltre, si pone enfasi sull'**Usabilità** dell'interfaccia, sulla **Portabilità** (per future espansioni ai dispositivi iOS) e sulla **Modifiability** per l'implementazione di future funzionalità aggiuntive a supporto dell'utenza target.

L'architettura logica di Tasky è basata su tre strati separati per ottimizzare la separazione dei compiti, la manutenibilità e la sicurezza: **Presentation Layer**, **Application Layer** (Logica di Business), **Data Access Layer (DAL)** per lo storage.

Viene utilizzato **MySQL** per la persistenza dei dati, scelto per la sua robustezza e il supporto

completo alle **transazioni ACID** (Atomicità, Coerenza, Isolamento, Durabilità). Il **DAL** gestisce l'accesso al database attraverso query parametrizzate, strategia che garantisce la corretta mitigazione del rischio di **SQL Injection**.

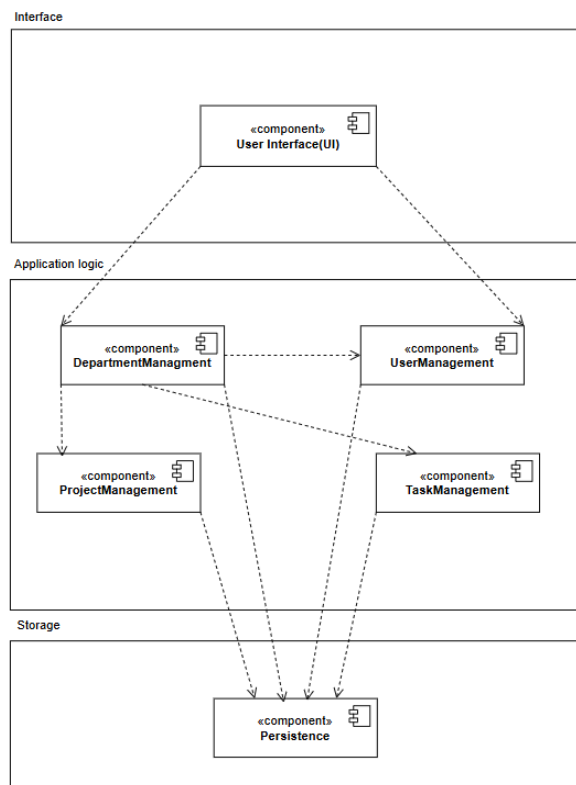
La sicurezza e l'autorizzazione sono rafforzate dal **modello RBAC**. Tutte le comunicazioni client-server sono interamente cifrate tramite **HTTPS/TLS**, e l'autenticazione è gestita tramite **password hashing forte** e **token bearer cifrati**.

È implementato un controllo asincrono che **monitora le scadenze** e impone automaticamente lo stato "Sospeso" sui Task che violano i termini definiti, preservando l'integrità dei dati temporali. L'intero sviluppo della nostra applicazione è rivolto alla risoluzione delle problematiche discusse nell'ambito delle **tecnologie correnti**, come ad esempio **Trello**. La concorrenza attuale verte spesso sul fornimento di **servizi prevalentemente a pagamento**, limitando l'esperienza utente degli attori coinvolti, i quali necessitano dell'acquisto di **piani premium** per contravvenire alle difficoltà legate al piano offerto gratuitamente.

Tasky propone una risoluzione a tali problematiche in maniera **affidabile** e **cost-effective**, fornendo un'esperienza utente significativa e incentrata sull'usabilità e sulla reliability dell'applicativo, garantendo che le funzionalità critiche di gestione e sicurezza siano intrinseche al sistema e accessibili, superando gli svantaggi legati alle attuali proposte di mercato. Il sistema opera in un'architettura progettata specificamente per gestire queste sfide, come presentato di seguito.

3.1 Overview

Tasky è una piattaforma di **Task Management aziendale mobile-first** basata su un'architettura **Three-Tier** implementata in Python/Flask. Il sistema garantisce l'efficienza operativa attraverso una rigorosa gestione dei ruoli (**RBAC** per Manager e Dipendenti) e un flusso di controllo basato su **richieste RESTful** (cifrate via **HTTPS/TLS**). La persistenza è assicurata da **MySQL** e **transazioni ACID**, con il **DAL** che utilizza **SQL puro parametrizzato** per la massima sicurezza contro la **SQL Injection**. Il progetto mira a superare i limiti delle piattaforme concorrenti offrendo intrinsecamente alta **Usabilità, Reliability, Supportability** e **Performance**, in linea con i Design Goals definiti.



3.2 Subsystem decomposition

I sottosistemi sono stati organizzati in **tre layer separati**:

- Il primo è dedicato **alla logica di presentazione**, che agisce come strato più esterno, rivolto all'utente, difatti contiene il **rendering dell'interfaccia**, acquisendo l'input dall'utente e gestendo la sua navigazione sul dispositivo mobile
- Il secondo è dedicato alla **logica di business** della nostra applicazione e rappresenta il cuore del sistema
- Il terzo è relativo alla **meccanica di persistenza**, attuata con il **DBMS** da noi scelto, ossia **MySQL**.

L'**Application Layer** è composto da specifiche componenti che incapsulano la logica di business per ogni dominio di interesse:

- **DepartmentManagement:** Gestisce le funzioni relative alla visualizzazione

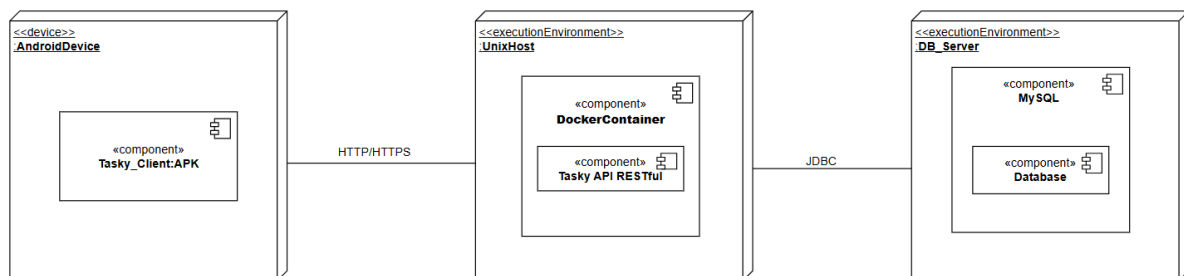
delle informazioni principali del dipartimento di lavoro, i membri che lo compongono e i rispettivi recapiti telefonici (sensibili) degli stessi.

- **UserManagement:** Gestisce gli account utente (Manager e Dipendenti), occupandosi delle fasi di **Autenticazione** (login e logout) e della gestione delle informazioni relative ai vari account aziendali.
- **TaskManagement:** Gestisce le funzioni riguardanti l'aggiunta, l'eliminazione, la modifica, la **sospensione** e la visualizzazione e la gestione complessiva dei **Task**.
- **ProjectManagement:** Gestisce le funzioni riguardanti l'aggiunta, la modifica, l'eliminazione e la visualizzazione o l'ulteriore gestione dei **Progetti**.

Infine, abbiamo la componente **Persistence**, che salva i prodotti degli altri sottosistemi all'interno del **database** e ne permette l'accesso quando richiesto.

3.3 Hardware/software mapping

L'applicazione Tasky prevede un'architettura basata su **tre livelli (Three-Tier Architecture)**, che richiede più nodi di distribuzione per le componenti software e hardware. La struttura architetturale proposta per il nostro sistema è stata scelta per massimizzare la scalabilità, la sicurezza e l'isolamento dei compiti, favorendo anche una futura portabilità del sistema.



Abbiamo 3 principali nodi:

- Per il **Presentation/Interface Layer** abbiamo l'**AndroidDevice** è costituito dalla **Tasky_Client** APK, la quale rappresenta l'applicazione mobile nativa Android, della quale in futuro vuole essere fornita anche una versione installabile per dispositivi iOS.
- Per l'**Application Logic Layer** abbiamo il Server, rappresentato da un **UnixHost (Server UNIX)**, composto di un **Docker Container**, contenente la logica di business (Python/Flask), il quale espone a sua volta la **Tasky API RESTful**.
- Infine, per lo **Storage Layer** abbiamo il **DB_Server**, il quale è composto del nostro DBMS, ossia **MySQL**, sistema di gestione dei dati persistenti all'interno del nostro database.

Questo schema riflette la logica three-tier in cui la logica applicativa funge da intermediario obbligatorio fra l'interfaccia utente e i dati, requisito per il nostro stile architetturale.

3.4 Persistent data management

La nostra applicazione Tasky si rifà all'utilizzo di un database relazionale per la persistenza dei dati di sistema. Il **DBMS** selezionato per la persistenza è **MySQL**. La scelta è motivata dai seguenti punti:

- È un **DBMS open-source** robusto.
- Offre **ottime prestazioni e sicurezza**.
- Supporta pienamente le **transazioni ACID**.

- È ampiamente **compatibile** con l'ecosistema **Python/Flask** (utilizzato per il backend). L'obiettivo principale è garantire **integrità e prestazioni** (RNF). MySQL è ottimizzato per carichi di lavoro misti e garantisce l'integrità referenziale richiesta dal modello di Tasky.

La persistenza sarà gestita mappando le classi principali del **Modello a Oggetti** (*Object Model*) in **tabelle relazionali**, preservando le associazioni tramite **chiavi primarie ed esterne**.

Le entità fondamentali riconosciute e protagoniste della persistenza nel database sono:

- **Progetto**: Caratterizzato da un nome, budget, descrizione, data di inizio e data di fine.
- **Dipartimento**: Caratterizzato da un nome e composto da dipendenti (il numero totale di dipendenti è un'informazione utile).
- **Task**: Caratterizzato da un nome, descrizione, stato, data di inizio e data di fine.
- **Dipendente**: Definito da nome, cognome, data nascita, sesso, recapito telefonico, email e password (aziendali) e un token di accesso.
- **Manager**: Definito da nome, cognome, data nascita, sesso, numero di telefono, anni lavorativi, email e password (aziendali) e token di accesso.

Per isolare la logica di business dalle specifiche SQL, Tasky implementa uno strato di **Data Access Layer (DAL)** esposto come un insieme di **repository functions**. Queste funzioni incapsulano le **operazioni CRUD e le query necessarie** per ogni entità (equivalente al pattern DAO/Repository, realizzato con funzioni procedurali piuttosto che con classi). Le funzioni del DAL eseguono query parametrizzate per prevenire SQL injection. Le operazioni che richiedono modifiche atomiche su più tabelle (es. inserimento utente + aggiornamento contatori di dipartimento) sono realizzate usando **transazioni esplicite**.

Vantaggi del DAL:

1. **Indipendenza dal DBMS**: Se in futuro si decidesse di migrare da MySQL a PostgreSQL, solo il DAL dovrebbe essere modificato.
2. **Sicurezza (RNF)**: La sicurezza del sistema è garantita contro attacchi comuni quali **SQL Injection** e **Unauthorized Access**, attraverso l'implementazione di **query parametrizzate** per le prime (effettuate dalle funzioni del DAL) e con la **cifratura e limite ai privilegi di accesso** per quanto riguarda le seconde.
3. **Transazionalità**: Le modifiche a più tabelle (es. creare un Progetto e assegnarlo a un Manager) saranno avvolte in **transazioni ACID** gestite dal DAL per garantire l'integrità.

Oltre alla persistenza dei dati applicativi nel database MySQL, il sistema **Tasky** implementerà un meccanismo di logging basato su file per tracciare le operazioni del sistema, gli eventi di sicurezza e gli errori di configurazione. Il logging è fondamentale per soddisfare i Requisiti Non Funzionali di **Supportabilità** (troubleshooting) e **Sicurezza** (audit trail).

3.5 Access control and security

3.5.1 Access Matrix

Ruolo	Visualizzazione task	Modifica Task	Aggiunta/Elimina Task	Sospensione Task	Visualizzazione progetti	Modifica Progetto	Aggiunta/Elimina Progetto	Visualizzazione membri dipartimento
Manager	V	V	V	V	V (solo assegnati)	V	V	V
Dipendente	V (solo assegnati)	V (limitata, solo assegnati)	X	X	X	X	X	X

Il sistema **Tasky** implementa un modello di accesso alle funzionalità e alle risorse basato su ruoli (**Role-Based Access Control - RBAC**). Questo approccio garantisce che le operazioni sugli oggetti (Task, Progetti, Dipartimenti) siano strettamente limitate in base al ruolo assegnato all'utente.

I principali ruoli e le relative capacità di accesso nell'applicazione sono:

Il **Dipendente** è l'esecutore dei task all'interno del proprio Dipartimento. I suoi permessi sono focalizzati sull'esecuzione e la rendicontazione del lavoro assegnato.

- **Lettura:** Ha la possibilità di **leggere** i **Task** che gli sono stati esplicitamente assegnati da un Manager, visualizzandoli in un'apposita schermata. Può inoltre **leggere** i **Progetti** a cui i suoi Task appartengono (nella sezione info relativa al task selezionato).
- **Modifica (Limitata):** Può **modificare** lo stato dei Task a lui assegnati, ma in maniera strettamente limitata: solo per segnalarne l'avanzamento (*In corso*) o il **completamento** (*Completato*).

Il **Manager** detiene la maggior parte del controllo amministrativo e gestionale all'interno dell'applicazione, supervisionando l'intero Dipartimento di cui è a capo.

- **Gestione Task e Progetti:** Ha il compito di **creare, modificare** (dettagli e stati), **aggiungere ed eliminare** sia i **Task** che i **Progetti** all'interno del proprio Dipartimento, in base alle esigenze e ai cambiamenti operativi.
- **Sospensione:** Può **sospendere un Task** specificando una **motivazione** (visibile al Dipendente) qualora la sua realizzazione sia temporaneamente impossibilitata.
- **Visualizzazione Dipartimento:** Ha la possibilità di visualizzare l'elenco completo dei Dipendenti che compongono il suo Dipartimento, inclusi i dati sensibili come il recapito telefonico aziendale.

3.5.2 Security Strategy

Il sistema **Tasky** adotta una strategia di sicurezza a più livelli, integrata in tutte le fasi di interazione (dalla persistenza alla comunicazione) per mitigare i principali rischi noti, in linea con i Requisiti Non Funzionali (RNF) di integrità e confidenzialità.

1. Sicurezza dei Dati (Data Security)

- **Prevenzione SQL Injection:** Il rischio di attacchi di iniezione SQL è mitigato in modo efficace attraverso l'utilizzo di **query parametrizzate** con *placeholder* all'interno dei moduli di accesso ai dati. Questo approccio separa chiaramente la logica SQL dai dati di input, prevenendo l'esecuzione di codice malevolo.
- **Password Hashing FORTE:** Le password degli utenti **non** sono mai salvate in chiaro nel database. Viene utilizzato un algoritmo di **hashing crittografico forte** e prima della memorizzazione.
- **Integrità Referenziale e Transazioni:** La coerenza e l'atomicità (proprietà ACID) delle operazioni multi-tabella sono garantite a livello di persistenza tramite l'uso di **chiavi esterne** (FOREIGN KEYS) nello schema del database e l'uso esplicito di transazioni nelle operazioni sensibili.

2. Controllo degli Accessi e Autorizzazione

- **Accesso non Autorizzato (RBAC):** La limitazione all'accesso non autorizzato è implementata tramite l'uso di **decorator di autorizzazione** dedicati. Questi controlli

verificano il ruolo e/o la proprietà della risorsa prima di consentire l'accesso agli *endpoint* sensibili dell'API.

- **Token Bearer:** Per l'autorizzazione post-autenticazione, il sistema utilizza un meccanismo di **token bearer**. Vengono forniti **token cifrati** per l'accesso, che devono essere inclusi in tutte le richieste successive, garantendo che solo le richieste autenticate e autorizzate possano raggiungere le risorse protette.

3. Igiene del Codice e Gestione degli Errori

- **Validazione Input:** La validazione dei payload JSON in ingresso è gestita centralmente tramite l'utilizzo di librerie di schemi come **marshmallow**. Questo processo valida la struttura e il tipo dei dati di input, riducendo significativamente il rischio derivante da input malformati o attacchi basati sulla manipolazione dei dati.
- **Minimizzazione Esposizione Sensibile:** Le risposte delle API sono progettate per non restituire mai dati sensibili non necessari. In particolare, le **password non vengono mai esposte** nelle risposte e il codice evita l'esposizione di altri campi sensibili nelle risposte standard, aderendo al principio del *need-to-know*.
- **Error Handling Centralizzato:** Gli errori applicativi e di sistema vengono trasformati in risposte API generiche, **impedendo la fuoriuscita di informazioni sensibili** come *stack trace* o dettagli interni del server.
- **Gestione dei Segreti:** I segreti critici (es. chiavi di crittografia, credenziali del database) sono gestiti in modo sicuro. Vengono utilizzati file di configurazione che sono esplicitamente esclusi dal sistema di controllo versione tramite l'uso di file `.gitignore`, prevenendo il *leak* involontario nel repository pubblico.

4. Audit e Tracciabilità

- **Logging di Sicurezza:** Un robusto sistema di **logging operativo e di sicurezza** è attivo per garantire l'audit, il *troubleshooting* e il rilevamento tempestivo di eventi sospetti. I log sono configurati per includere: eventi di autenticazione (login successo/fallimento), operazioni amministrative critiche, modifiche ai dati sensibili ed errori applicativi.
- **Protezione dei Log:** Tutti i log sono trattati al momento della scrittura per **evitare l'esposizione diretta di password o token**, garantendo che i dati sensibili rimangano protetti anche all'interno dei file di log.

3.6 Global software control

La nostra applicazione, ossia Tasky, prevede che il controllo software a livello di sistema venga implementato secondo un approccio centralizzato e guidato da eventi (Event-driven) sul lato server, con due modalità operative principali:

1. **Controllo Dominante:** Richiesta/Risposta, tramite la quale viene governata l'interazione sincrona tra il Client (AndroidDevice) ed il Server (UnixHost). Il server in questo caso agisce da dispatcher(router Flask). Ogni richiesta API in arrivo (evento HTTP/RESTful) infatti è gestita in modo sincrono, dunque il presentation layer genera l'evento, l'application Logic Layer riceve, autentica e autorizza (RBAC) la richiesta e l'esecuzione procede fino al Data Access Layer (DAL) che interagisce con MySQL.
2. **Controllo Asincrono:** Basato sul tempo (Time-Triggered), dove un servizio schedulato sul server si attiva ad intervalli regolari (effettuato alle ore 00:00 di ogni giorno), eseguendo in modo asincrono la logica per monitorare le scadenze violate ed imporre lo stato "Sospeso" ai task, come definito nel RAD.

3.6.1 Concurrency Management

La gestione della concorrenza è fondamentale soprattutto per i RNF di Performance e di Affidabilità

del sistema. Abbiamo:

- **Threading Parallelo:** il server applicativo alloca un **nuovo thread per ciascuna richiesta** in ingresso (evento HTTP/RESTful). Questo modello consente la gestione parallela di centinaia di richieste utente, migliorando la reattività e il throughput del sistema.
- **Sincronizzazione (ACID e Locking):** La responsabilità di prevenire le **race condition** e garantire l'integrità dei dati (RNF) ricade su:

1. DAL e Logica Applicativa (S2/S3): Le operazioni che modificano più tabelle o richiedono la lettura di uno stato prima della scrittura devono essere gestite all'interno di **transazioni esplicite** con `BEGIN TRANSACTION` e `COMMIT/ROLLBACK`.

2. DBMS (MySQL): Per prevenire **lost updates** (due utenti modificano lo stesso Task), l'accesso concorrente ai dati persistenti è gestito utilizzando le proprietà **ACID** del DBMS e, se necessario per le operazioni critiche, tramite **blocchi espliciti a livello di riga** (`SELECT ... FOR UPDATE` nell'SQL puro), assicurando che solo un thread alla volta possa modificare quel particolare record.

In sintesi, il Controllo Software Globale definisce un progetto strategico dinamico in cui il controllo centralizzato dal server è bilanciato dall'efficienza del **multithreading** e dalla rigorosa sincronizzazione garantita dal DAL e dal DBMS.

3.7 Boundary conditions

- **Startup sistema**

Nome caso d'uso		Startup sistema	
Attori partecipanti		Sistemista	
Flussi di eventi	Sistemista	Tasky	
	1. Il sistemista scrive in console docker compose up (--build se è la prima volta che deve essere eseguito)		
		2. Il sistema effettua l'inizializzazione delle tabelle nel database e viene avviata l'API	
Pre-Condizioni		Il sistemista è connesso tramite una shell al server che ospita l'applicazione	
Post-Condizioni		L'applicazione è raggiungibile da qualsiasi dispositivo Android che abbia installata l'applicazione di UI ed è connesso alla rete.	

- **Shut-down sistema**

Nome caso d'uso		Shut down sistema	
Attori partecipanti		Sistemista	
Flussi di eventi	Sistemista	Tasky	

	1. Il sistemista scrive in console docker compose down	
		2. Il sistema effettua la chiusura di tutti i servizi gestiti in startup (con database ancora persistente)
Pre-Condizioni		Il sistemista è connesso tramite una shell al server che ospita l'applicazione
Post-Condizioni		L'applicazione è raggiungibile da qualsiasi dispositivo Android che abbia installata l'applicazione di UI ed è connesso alla rete.

- **Fallimento**

Tasky può incorrere a diversi casi di fallimento, riguardanti sia l'hardware che il software:

- **Fallimenti hardware:** possono riguardare lo spazio persistente esaurito. Il sistema non prevede alcuna strategia di backup e ripristino dei dati.
- **Fallimenti nell'ambiente di esecuzione:** problematiche relative all'ambiente di esecuzione o cali di energie con possibili cause ambientali. Il sistema non prevede alcuna strategia che ne garantisca l'operabilità in questo tipo di condizione.
- **Fallimenti software:** può riguardare l'ambito persistente, dunque il database che può risultare corrotto o non raggiungibile.

4. Subsystem services

Servizio	Descrizione
Autenticazione	Fornisce la possibilità all'utente di poter accedere o uscire all'applicazione, gestendo il login e il logout di quest'ultimo.

Le operazioni coinvolte sono :

- Login
- Logout

Servizio	Descrizione
Gestione task dipendente	Fornisce la possibilità al dipendente di poter visualizzare i task di cui è responsabile in ambito di realizzazione e di poterne modificare lo stato nel caso in cui ne esegua il completamento.

Le operazioni coinvolte sono :

- Visualizzazione tasks
- Cambiamento stato task

Servizio	Descrizione
Gestione Task manager	Fornisce la possibilità al manager di poter gestire in toto i tasks assegnati ai membri del proprio dipartimento. Egli potrà visualizzare la lista dei tasks dei propri progetti, aggiungere o eliminare dei tasks, modificarne le informazioni peculiari o sospenderli in caso di impossibilità di realizzazione, con annessa motivazione

Le operazioni coinvolte sono :

- Aggiunta task
- Eliminazione task
- Modifica Task
- Sospensione task
- Visualizzazione dei tasks

Servizio	Descrizione
Gestione Progetti manager	Fornisce la possibilità al manager di poter gestire in toto i progetti di cui è manager. Egli potrà visualizzare la lista dei propri progetti, aggiungere o eliminarne e modificarne le informazioni peculiari.

Le operazioni coinvolte sono :

- Aggiunta Progetto
- Eliminazione Progetto
- Modifica Progetto
- Visualizzazione dei progetti

Servizio	Descrizione
Gestione Dipartimento manager	Fornisce la possibilità al manager di poter tenere traccia dei membri del proprio dipartimento, quindi dei dipendenti, visualizzandone in caso necessario anche il relativo recapito telefonico

Le operazioni coinvolte sono :

- Visualizza recapito telefonico
- Visualizza membri

5. Glossary

TERMINE	DEFINIZIONE
Tasky	Applicazione Task Manager aziendale
Manager	Figura responsabile della creazione, delegazione e supervisione dei tasks all'interno dell'azienda; ricopre un ruolo amministrativo e di coordinamento dei progetti.
Dipendente	Utente che esegue i tasks assegnati da un manager ed è autorizzato a visualizzarli. Ha inoltre la responsabilità di aggiornare lo stato di avanzamento dei seguenti.
Progetto	Insieme strutturato di tasks assegnati ad un dipartimento, definito da una Data di Inizio e una Data di Fine, un Budget, una Descrizione ed un nome.
Dipartimento	Insieme di dipendenti che lavorano su specifiche tipologie di progetto, supervisionato da un manager
Task	È l'unità minima di lavoro che deve essere completata da un Dipendente all'interno di un Progetto
Sistemista	Persona abilitata alla gestione e amministrazione del sistema, responsabile dell'avvio e dello spegnimento (startup/shutdown) dell'infrastruttura.
API RESTful	Un'interfaccia di programmazione applicativa (API) che aderisce ai principi di progettazione dell'architettura REST (Representational State Transfer) .
HTTPS	Versione sicura di http con crittografia SSL/TLS per proteggere le comunicazioni
Department Management	Funzionalità per gestire e visualizzare un dipartimento e i membri contenuti al suo interno
Task Management	Funzionalità per aggiungere, eliminare, modificare, sospendere e visualizzare unità di lavoro, ossia i tasks
Project Management	Funzionalità per aggiungere, eliminare, modificare e visualizzare i progetti di uno

	specifico dipartimento
User Management	Unità di gestione degli account utente e del servizio di autenticazione
Event-Driven Model	Architettura software le cui operazioni sono scaturite e attivate da eventi, come input o cambiamenti di stato
Bearer Token	Meccanismo di autenticazione in cui chi possiede un token valido può accedere alle risorse autorizzate.
Role-Based Access Control	Modello di controllo degli accessi basato sui ruoli associati ai vari utenti
Tasky API RESTful	Interfaccia RESTful esposta dal nostro server di backend realizzata tramite il framework Flask
Three Tier Architecture	un'architettura logica e fisica client-server ben definita in cui l'interfaccia utente, la logica di business (o logica funzionale) e le funzionalità di gestione dei dati sono sviluppate e mantenute come moduli indipendenti , spesso su piattaforme separate.